

ĐẠI HỌC QUỐC GIA THÀNH PHỐ HỒ CHÍ MINH
TRƯỜNG ĐẠI HỌC BÁCH KHOA
KHOA KHOA HỌC VÀ KỸ THUẬT MÁY TÍNH



BÁO CÁO

Mạng Máy Tính

WEB PROXY VỚI XÁC THỰC VÀ ỨNG DỤNG CHAT HYBRID

Lớp: [L06] – HỌC KỲ 251
Giảng viên hướng dẫn: Bùi Xuân Giang

Sinh viên thực hiện:

Phan Lê Thiên	2313221
Nguyễn Đức Thịnh	2313284
Trần Hồng Quảng	2312816
Trần Hữu Nam	2312196

TP. Hồ Chí Minh, ngày 13 tháng 11 năm 2025

Mục lục

1	Mục tiêu	2
2	Phân tích yêu cầu	2
2.1	Task 1: Web Proxy và Xác thực Session	2
2.2	Task 2: Ứng Dụng Chat Hybrid (Client-Server + P2P)	2
3	Kiến trúc hệ thống (Phiên bản Hợp nhất)	3
4	Triển khai Task 1: Quản lý Phiên (Session)	3
4.1	Module <code>request.py</code> (Bộ phân tích HTTP)	3
4.2	Module <code>response.py</code> (Bộ xây dựng phản hồi)	4
4.3	Nâng cấp Unified Server (<code>start_sampleapp.py</code>)	4
5	Luồng xử lý Task 1 (Phiên bản Session)	4
6	Triển khai Task 2: Ứng Dụng Chat Hybrid (Web UI)	5
6.1	Kiến trúc Hybrid trên Web	5
6.2	Hiện thực Giai đoạn Client-Server (Chat Nhóm)	5
6.2.1	Unified Server (<code>start_sampleapp.py</code>)	5
6.2.2	Frontend (<code>chat.js</code>)	5
6.3	Hiện thực Giai đoạn P2P (Chat 1-với-1 WebRTC)	6
6.3.1	Unified Server (Vai trò Signaling)	6
6.3.2	Frontend (<code>chat.js</code> - Logic P2P)	6
6.4	Hiện thực Channel Management (UI và Tính năng)	6
7	Kết quả thực hiện Task 2 – Hình ảnh minh họa	7
8	Tích hợp Task 1 và Task 2 (Combine)	8
9	Kết luận	8

I. Mục tiêu

Assignment 1 yêu cầu sinh viên triển khai hai hệ thống mạng phân tán độc lập, trong đó Task 1 tập trung vào việc xây dựng một hệ thống Web Proxy hoạt động theo mô hình client-server với cơ chế định tuyến dựa trên hostname và tích hợp xác thực người dùng thông qua form đăng nhập, được chia thành Task 1A (proxy) và Task 1B (xác thực). Task 2 yêu cầu phát triển một ứng dụng chat hybrid kết hợp client-server và peer-to-peer, hỗ trợ quản lý kênh và đồng bộ hóa tin nhắn theo thời gian thực.

II. Phân tích yêu cầu

1. Task 1: Web Proxy và Xác thực Session

Yêu cầu của Task 1 là xây dựng một hệ thống xác thực dựa trên cookie. Trong phiên bản nâng cấp, hệ thống này không chỉ cấp một cookie `auth=true` chung chung, mà đã được phát triển thành một hệ thống **Quản lý Phiên (Session Management)** thực thụ, có khả năng phân biệt nhiều người dùng khác nhau.

- **Đăng nhập Đa người dùng:** Hệ thống phải có khả năng xác thực nhiều tài khoản (ví dụ: `userA`, `userB`, `userC`).
- **Cấp Session ID:** Thay vì cookie tĩnh, server phải tạo một `session_id` ngẫu nhiên, duy nhất cho mỗi lần đăng nhập thành công và gửi về client qua `Set-Cookie`.
- **Kiểm soát Truy cập (Access Control):** Các tài nguyên được bảo vệ (như `/chat.html`) phải được kiểm tra. Nếu client không gửi `session_id` hợp lệ (đã được server lưu trữ), hệ thống phải trả về lỗi 401 Unauthorized.

2. Task 2: Ứng Dụng Chat Hybrid (Client-Server + P2P)

Yêu cầu của Task 2 là xây dựng một ứng dụng chat hybrid hỗ trợ cả hai mô hình, đồng thời đáp ứng các yêu cầu về giao diện (Channel Management).

- **Initialization Phase (Client-Server):** Đây là giai đoạn "mai mối". Peer phải kết nối đến một server trung tâm (Tracker) để đăng ký và lấy danh sách các peer khác.
- **Peer Chatting Phase (P2P):** Sau khi có danh sách, các peer phải có khả năng thiết lập kết nối P2P trực tiếp để trao đổi tin nhắn mà không cần đi qua server.
- **Channel Management (UI):** Ứng dụng phải có giao diện web thân thiện, hỗ trợ:
 - **Channel listing:** Hiển thị danh sách kênh (ví dụ: kênh chung) và danh sách người dùng đang hoạt động.
 - **Message display:** Khung chat có thể cuộn (scrollable).
 - **Message submission:** Có ô nhập liệu và nút gửi.
 - **Notification system:** Có thông báo tin nhắn mới (ví dụ: "chấm đỏ").
- **Combine (Hợp nhất):** Logic xác thực của Task 1 phải được dùng để bảo vệ

các API của Task 2.

III. Kiến trúc hệ thống (Phiên bản Hợp nhất)

Kiến trúc hiện tại đã được nâng cấp thành một mô hình "Unified Server" (Máy chủ Hợp nhất) linh hoạt, sử dụng `start_sampleapp.py` làm backend duy nhất.

- **Proxy Server (`start_proxy.py`):** Vẫn là cửa ngõ tại port 8080. Đọc `proxy.conf` để chuyển tiếp mọi request đến Unified Server tại port 9000.
- **Unified Server (`start_sampleapp.py`):** Chạy tại port 9000, là trái tim của hệ thống. Nó được xây dựng trên framework `WeApRous` và định nghĩa toàn bộ logic nghiệp vụ. Nó đóng 3 vai trò:
 1. **Auth Server (Task 1):** Quản lý `USER_DB`, `SESSIONS` và xử lý `POST /login` để cấp cookie `session_id`.
 2. **Chat Relay Server (Task 2 - C/S):** Quản lý kênh chat chung (`CHANNEL_HISTORY`), nhận và lưu tin nhắn cho kênh `#general`.
 3. **P2P Signaling Server (Task 2 - P2P):** Làm "bưu điện" (mai mối) cho các kết nối WebRTC, chuyển tiếp tín hiệu Offer/Answer/ICE qua các API `/send-signal` và `/get-signals`.
- **WeApRous Framework (`weaprou.py`):** Cung cấp decorator `@app.route` để đăng ký các hàm xử lý (handler) cho các đường dẫn (path) và phương thức (method) cụ thể.
- **Backend Module (`backend.py`):** Là "động cơ" cấp thấp, chịu trách nhiệm tạo socket server, lắng nghe kết nối và tạo luồng (thread) mới cho mỗi client.
- **HttpAdapter Module (`httpadapter.py`):** Đóng vai trò **Bộ điều phối (Dispatcher)**.
 - Đọc toàn bộ request từ socket một cách an toàn (sử dụng `Content-Length` để đọc hết body, khắc phục lỗi `recv()` thiếu dữ liệu).
 - Gọi `request.py` để parse dữ liệu thô.
 - **Ưu tiên 1 (Hook):** Kiểm tra nếu `req.hook` tồn tại (do `start_sampleapp.py` định nghĩa), nó sẽ thực thi hook đó (ví dụ: `web_login`, `send_signal`) và kết thúc.
 - **Ưu tiên 2 (File tĩnh & Bảo vệ):** Nếu không có hook, nó sẽ xử lý các request `GET`. Các trang trong `protected_pages` (như `/chat.html`) sẽ bị kiểm tra cookie. Các file tĩnh công khai (như `/login.html`) được phục vụ tự do.

IV. Triển khai Task 1: Quản lý Phiên (Session)

Task 1 được nâng cấp từ xác thực đơn giản lên Quản lý Phiên đa người dùng.

1. Module `request.py` (Bộ phân tích HTTP)

Lớp `Request` chịu trách nhiệm biến chuỗi request thô thành đối tượng Python.

- **Hàm `prepare()`:** Sử dụng `\r\n\r\n` để tách biệt `header_section` và `body_section`.
- **Hàm `prepare_cookies()`:** Phân tích header `Cookie` (ví dụ: `"session_id=abc..."`) thành một dictionary.
- **Hàm `prepare_body()`:** Gán `body_section` vào `self.body` và cũng phân tích `application/x-www-form-urlencoded` (cho form login cũ) vào `self.form`.

2. Module `response.py` (Bộ xây dựng phản hồi)

Lớp `Response` chịu trách nhiệm xây dựng phản hồi HTTP chuẩn.

- **Hàm `build_response_header()`:** Tự động duyệt `self.cookies` và thêm các header `Set-Cookie: session_id=...; Path=/` khi được yêu cầu.
- **Hàm `build_unauthorized()`:** Hàm tiện ích tạo nhanh phản hồi 401.

3. Nâng cấp Unified Server (`start_sampleapp.py`)

Đây là nơi logic Task 1 thực sự được định nghĩa, thay vì trong `httpadapter`.

- **CSDL User & Session:** Khởi tạo 3 CSDL giả lập: `USER_DB` (lưu 3 tài khoản), `SESSIONS` (lưu các `session_id` đang hoạt động), và `ACTIVE_USERS_WEB` (lưu "nhịp tim" nhằm xác định xem users nào còn active).
- **Hàm `web_login()`:** Được đăng ký tại `POST /login`. Hàm này:
 1. Nhận JSON `{username, password}` từ `login.js`.
 2. Kiểm tra với `USER_DB`.
 3. Nếu đúng, tạo một `session_id` ngẫu nhiên bằng `uuid.uuid4()`.
 4. Lưu vào CSDL session: `SESSIONS[session_id] = username`.
 5. Gán `resp.cookies['session_id'] = session_id` và trả về 200 OK.
- **Hàm `get_username_from_session(req)`:** Hàm hỗ trợ (helper) dùng để kiểm tra cookie trong mọi API được bảo vệ.

V. Luồng xử lý Task 1 (Phiên bản Session)

Khi truy cập GET `/chat.html` (Chưa đăng nhập):

1. Client → Proxy → Unified Server.
2. `httpadapter` nhận request. `req.hook` không tồn tại.
3. Logic rơi xuống Lớp 3 (File tĩnh). `req.path (/chat.html)` nằm trong `protected_pages`.
4. `httpadapter` kiểm tra cookie → không thấy → gọi `resp.build_unauthorized()` và trả về 401. Trình duyệt hiển thị lỗi.

Khi đăng nhập POST `/login` (từ `login.js`):

1. Client → Proxy → Unified Server.
2. `httpadapter` nhận request. `req.hook` TỒN TẠI (cho `POST /login`).

3. Logic Lớp 1 (Hook) được kích hoạt. Hàm `web_login` được gọi.
4. `web_login` xác thực `userA/password` thành công, tạo `session_id` (ví dụ: `abc-123`), lưu vào `SESSIONS` và gán vào `resp.cookies`.
5. `httpadapter` gửi phản hồi 200 OK (chứa header `Set-Cookie: session_id=abc-123...`).

Khi truy cập GET /chat.html (Sau khi đăng nhập):

1. `login.js` gọi `window.location.href = '/chat'`.
2. Client (gửi kèm cookie `session_id=abc-123`) → Proxy → Unified Server.
3. `httpadapter` nhận request. `req.hook` TỒN TẠI (cho GET /chat).
4. Hàm `chat_page` được gọi, đọc file `www/chat.html` và gửi về. Trình duyệt hiển thị giao diện chat.

VI. Triển khai Task 2: Ứng Dụng Chat Hybrid (Web UI)

Hệ thống được thiết kế để hỗ trợ cả hai mô hình chat (Relay và P2P) trên cùng một giao diện Web.

1. Kiến trúc Hybrid trên Web

Do giới hạn bảo mật của trình duyệt (không thể mở server socket), chúng ta đã triển khai:

- **Mô hình Client-Server (Relay):** Áp dụng cho các kênh chat chung (public chat room như `#general`). Tin nhắn `A` → Server → `B`. Server lưu trữ lịch sử chat.
- **Mô hình Peer-to-Peer (WebRTC):** Áp dụng cho chat riêng 1-với-1. Tin nhắn `A` → `B` (trực tiếp). Server chỉ đóng vai trò "mai mối" (Signaling).

2. Hiện thực Giai đoạn Client-Server (Chat Nhóm)

a. Unified Server (*start_sampleapp.py*)

- **CSDL Kênh:** Khởi tạo `CHANNEL_HISTORY = {"general": []}` để lưu tin nhắn kênh chung.
- **API POST /send-message-channel:** Khi `chat.js` gọi, server nhận tin nhắn, lấy `username` từ session, và lưu tin nhắn vào `CHANNEL_HISTORY['general']`.
- **API POST /get-messages-channel:** Khi `chat.js` gọi (kèm tên kênh), server trả về toàn bộ lịch sử chat của kênh đó.

b. Frontend (*chat.js*)

- **Polling (Lấy tin):** Khi user đang ở kênh `#general`, `setInterval` liên tục gọi `/get-messages-channel` mỗi 2 giây để lấy tin nhắn mới và vẽ lại khung chat.

- **Gửi tin:** Khi nhấn "Send" (trong kênh #general), chat.js gọi POST /send-message-channel (gửi tin lên server).

3. Hiện thực Giai đoạn P2P (Chat 1-với-1 WebRTC)

a. Unified Server (Vai trò Signaling)

Server đóng vai trò "bưu điện" để chuyển tiếp tín hiệu "bắt tay" P2P.

- **CSDL Hòm thư:** Khởi tạo SIGNAL_MAILBOX = {}.
- **API POST /send-signal:** Khi User A muốn kết nối User B, A gửi "lời mời" (Offer) lên API này. Server nhận, tìm hòm thư của B và thả "lời mời" vào: SIGNAL_MAILBOX['userB'].ap
- **API GET /get-signals:** User B liên tục gọi API này (Polling). Khi thấy có thư mới, Server trả thư về cho B và xóa thư khỏi hòm thư.

b. Frontend (chat.js - Logic P2P)

Đây là phần phức tạp nhất, sử dụng API RTCPeerConnection của trình duyệt.

- **Khởi tạo (Initiate):** Khi User A (đang xem chat.html) click nút "Chat P2P" cạnh tên User B:
 1. A tạo pc = new RTCPeerConnection(stunConfig).
 2. A tạo "đường ống" chat: dc = pc.createDataChannel("chat").
 3. A tạo "lời mời": offer = await pc.createOffer().
 4. A gửi Offer này lên Server qua POST /send-signal (để gửi cho B).
- **Chấp nhận (Accept):**
 1. User B (đang ở chat.html) poll thấy "Offer" của A (từ GET /get-signals).
 2. B tạo pc của mình, nhận Offer, và tạo "lời chấp nhận": answer = await pc.createAnswer().
 3. B gửi Answer này lên Server qua POST /send-signal (để gửi cho A).
- **Kết nối:** Cả A và B trao đổi thêm "chỉ đường" (ICE Candidates) qua Server. Khi thành công, kết nối P2P được thiết lập.
- **Chat P2P:** Khi A gửi tin cho B, chat.js gọi dataChannel.send(message). Tin nhắn đi thẳng P2P, **không đi qua Server**.
- **Nhận tin P2P:** chat.js của B nhận tin qua sự kiện dataChannel.onmessage và hiển thị lên màn hình.

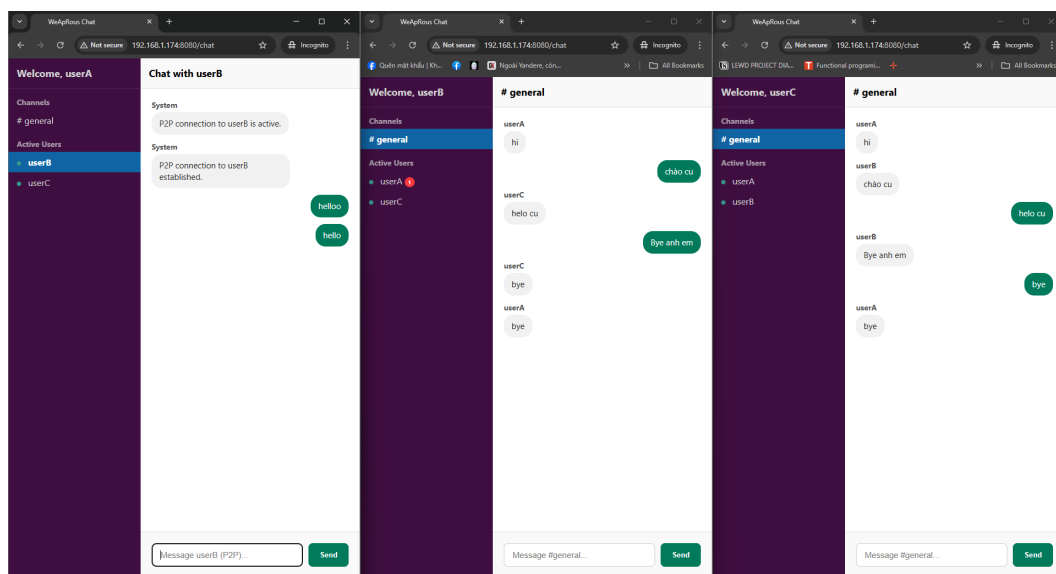
4. Hiện thực Channel Management (UI và Tính năng)

- **Quản lý User (Heartbeat):** Để biết ai đang online, chat.js gửi POST /heartbeat mỗi 2 giây. Server (start_sampleapp.py) có một luồng dọn dẹp, xóa bất kỳ user nào không gửi "nhịp tim" quá 7 giây, làm họ "biến mất" khỏi danh sách Active Users.
- **Lưu trữ Chat P2P (Client-side):** Để giữ lại lịch sử chat riêng, chat.js lưu

trữ tin nhắn P2P vào một biến CSDL cục bộ (`clientChatHistory`). Khi user click qua lại giữa các kênh, lịch sử chat P2P không bị mất.

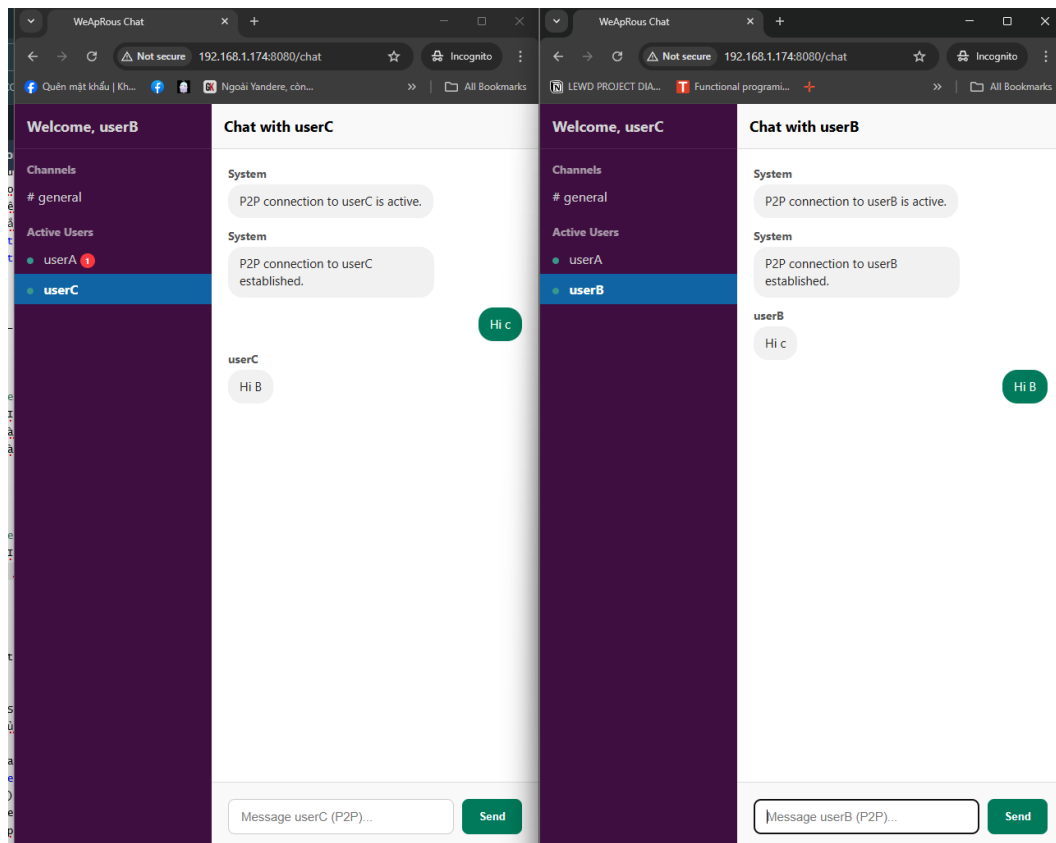
- **Thông báo (Chấm đỏ):** Khi tin nhắn (cả P2P và Kênh) đến, `chat.js` kiểm tra xem user có đang xem kênh/user đó không. Nếu không, nó tăng bộ đếm `unreadCounts` và `updateSidebar()` sẽ vẽ một "chấm đỏ" thông báo bên cạnh tên.
- **Giao diện Tin nhắn (Trái/Phải):** Sử dụng CSS `align-self: flex-end` cho tin nhắn `.sent` (gửi đi) và `align-self: flex-start` cho tin nhắn `.received` (nhận về) để tạo giao diện chat chuẩn hiện đại.

VII. Kết quả thực hiện Task 2 – Hình ảnh minh họa



Thay ảnh demo chat Web UI (A, B, C chat) vào đây

Hình 1. Giao diện Web Chat hoàn chỉnh. Bên trái là Kênh chat chung và Active Users (có chấm đỏ thông báo). Bên phải là khung chat P2P với tin nhắn trái/phải.



Thay ảnh demo chat Web UI vs Terminal vào đây

Hình 2. người dùng B và C kết nối với nhau để trò chuyện riêng với nhau.

VIII. Tích hợp Task 1 và Task 2 (Combine)

Yêu cầu cuối cùng là hợp nhất hai task. Điều này đã được hoàn thành triệt để bằng cách:

1. **Sử dụng Unified Server:** Chạy `start_sampleapp.py` là máy chủ duy nhất, chứa logic của cả Task 1 (Session) và Task 2 (Relay + Signaling).
2. **Bảo vệ API của Task 2:** Mọi API quan trọng của Task 2 (như `/get-sidebar-data`, `/send-message-channel`, `/send-signal`, `/get-signals`) đều được bảo vệ bằng hàm `get_username_from_session(req)`. Nếu client (Web UI) không gửi kèm cookie `session_id` hợp lệ, server sẽ trả về lỗi 401 Unauthorized.

Hệ thống cuối cùng vừa là một web server có xác thực đa người dùng, vừa là một hệ thống chat Hybrid hỗ trợ cả chat nhóm (Relay) và chat riêng (WebRTC P2P), đáp ứng đầy đủ yêu cầu của đề bài.

IX. Kết luận

Thông qua hai bài tập, nhóm đã xây dựng thành công một hệ thống mạng phức tạp từ các module cơ bản sử dụng socket. Task 1 đã cung cấp nền tảng về xử lý HTTP, parsing, và Quản lý Phiên (Session) đa người dùng. Task 2 đã áp dụng các kiến thức này để xây dựng một ứng dụng Hybrid Chat thực thụ, hỗ trợ cả hai mô hình giao tiếp:

Client-Server (Relay) cho chat nhóm công khai và Peer-to-Peer (WebRTC) cho chat riêng tư, đồng thời tích hợp một giao diện Web UI (Channel Management) hoàn chỉnh. Toàn bộ hệ thống cuối cùng đã được hợp nhất, sử dụng cơ chế xác thực của Task 1 để bảo vệ các tài nguyên của Task 2.